

Тема урока: модуль turtle

1. Черепашья графика
2. Модуль `turtle`
3. Рисование отрезков прямой
4. Поворот черепашки
5. Установка углового направления черепашки
6. Изменение внешнего вида черепашки

Черепашья графика

В конце 1960-х годов преподаватель Массачусетского технологического института (MIT) [Сеймур Пейперт](#) начал использовать роботизированную "черепашку" для обучения студентов программированию.

Черепашка была связана с компьютером, на котором обучаемый мог вводить команды, побуждающие черепашку перемещаться. У черепашки имелось перо, которое можно было поднимать и опускать. Ее можно было положить на лист бумаги и, программируя движение, создать рисунок.



Python имеет систему черепашьей графики, имитирующую эту роботизированную черепашку. Система выводит на экран небольшой курсор (черепашку). Перемещение черепашки по экрану рисует линии и геометрические фигуры. Для этого используется модуль `turtle`.

Модуль turtle

Система черепашьей графики не встроена в интерпретатор Python и хранится в модуле `turtle`. В начале программы с использованием черепашьей графики пишут инструкцию импорта этого модуля:

```
import turtle
```

Рисование отрезков прямой

В результате команды `turtle.showturtle()` появляется графическое окно с черепашкой. Черепашка первоначально расположена в центре графического окна, "холста". Выглядит она как стрелка с острием на месте головы. Если дать черепашке команду двигаться вперед, она переместится в направлении, указываемом стрелкой.



По умолчанию на старте черепашка смотрит на восток.

Для перемещения черепашки вперед на n пикселей применяется команда `turtle.forward(n)`. Например, команда `turtle.forward(100)` переместит черепашку вперед на 100 пикселей.

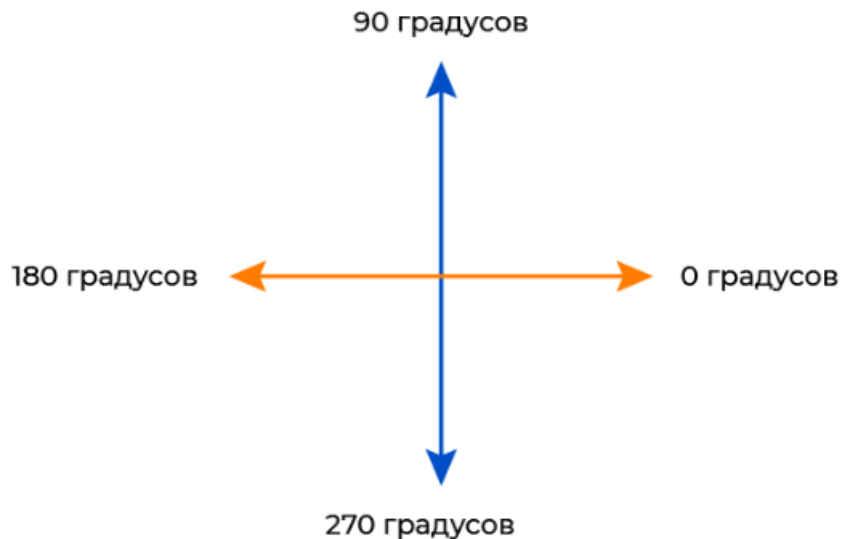
Для перемещения черепашки назад на n пикселей применяется команда `turtle.backward(n)`. Например, команда `turtle.backward(250)` переместит черепашку назад на 250 пикселей.

```
import turtle
turtle.showturtle()
turtle.forward(100) #Направо
turtle.backward(250) #Налево
```



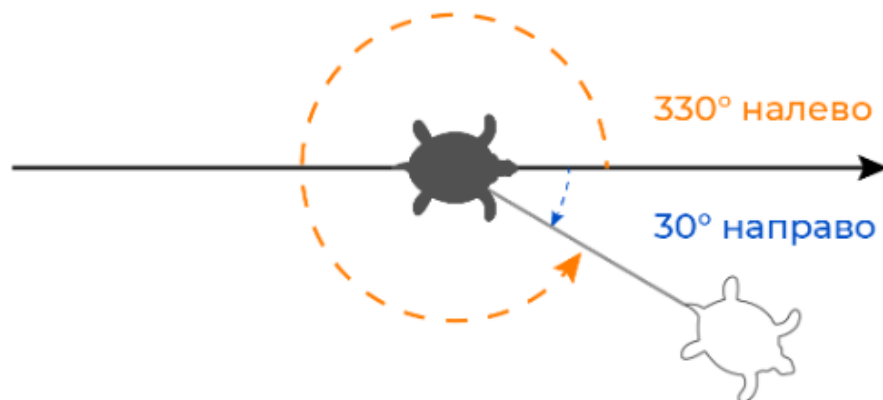
Поворот черепашки

Когда черепашка появляется, она по умолчанию направлена на восток, то есть вправо, или под углом 0° .



При помощи команд `turtle.right()` и `turtle.left()` можно повернуть черепаху:

- команда `turtle.right(angle)` поворачивает черепаху вправо на `angle` градусов;
- команда `turtle.left(angle)` поворачивает черепаху влево на `angle` градусов.



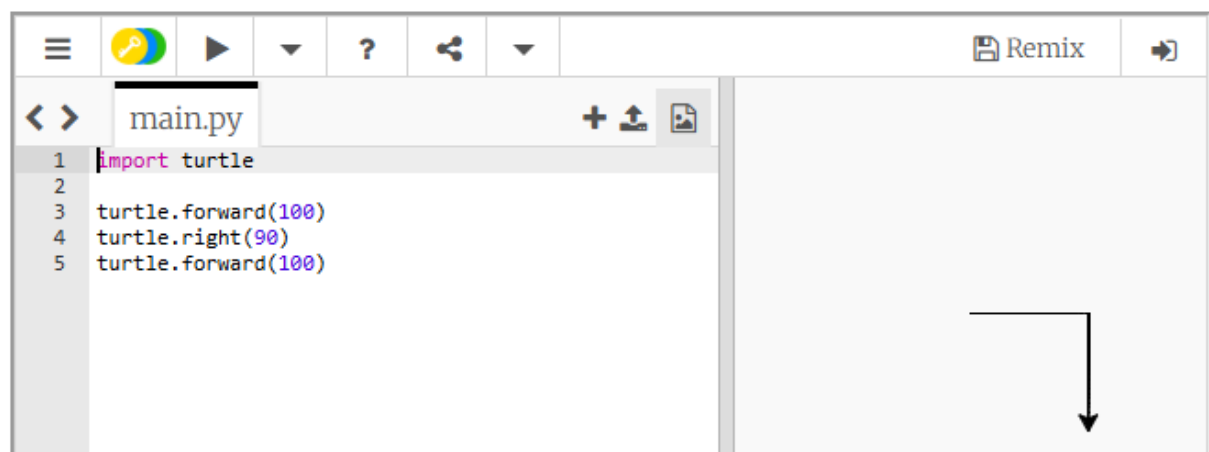
Рассмотрим программный код:

```
import turtle

turtle.forward(100)
turtle.right(90)
turtle.forward(100)
```

Такой код перемещает черепаху на 100 пикселей вперед. Далее он поворачивает черепаху на 90° вправо (черепашка будет смотреть вниз). Затем он перемещает черепаху на 100 пикселей вперед.

Такой код перемещает черепаху на 100 пикселей вперед. Далее он поворачивает черепаху на 90° вправо (черепашка будет смотреть вниз). Затем он перемещает черепаху на 100 пикселей вперед.

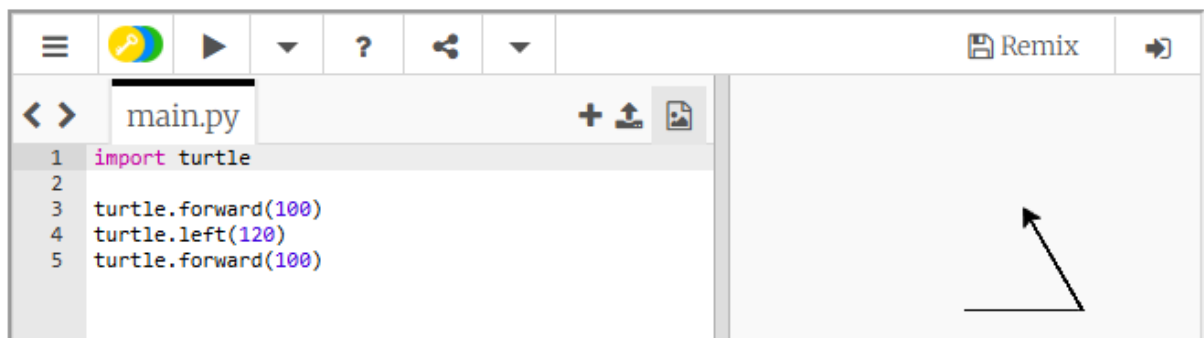


Рассмотрим программный код с использованием команды `turtle.left()` :

```
import turtle

turtle.forward(100)
turtle.left(120)
turtle.forward(100)
```

Такой код сначала перемещает черепашку на 100 пикселей вперед. Затем он поворачивает черепашку на 120° влево (черепашка будет смотреть на северо-запад). И далее он перемещает черепашку на 100 пикселей вперед.



Команды `turtle.right()` и `turtle.left()` поворачивают черепашку на указанный угол от ее направления в этот момент, а не от исходного. К примеру, текущее угловое направление составляет 90° , строго на север. Если ввести команду `turtle.left(20)`, то черепашка повернется влево на 20° . Ее новое угловое направление составит 110° , по отношению к начальному положению, когда угол равен 0° .

В качестве еще одного примера рассмотрим код:

```
import turtle

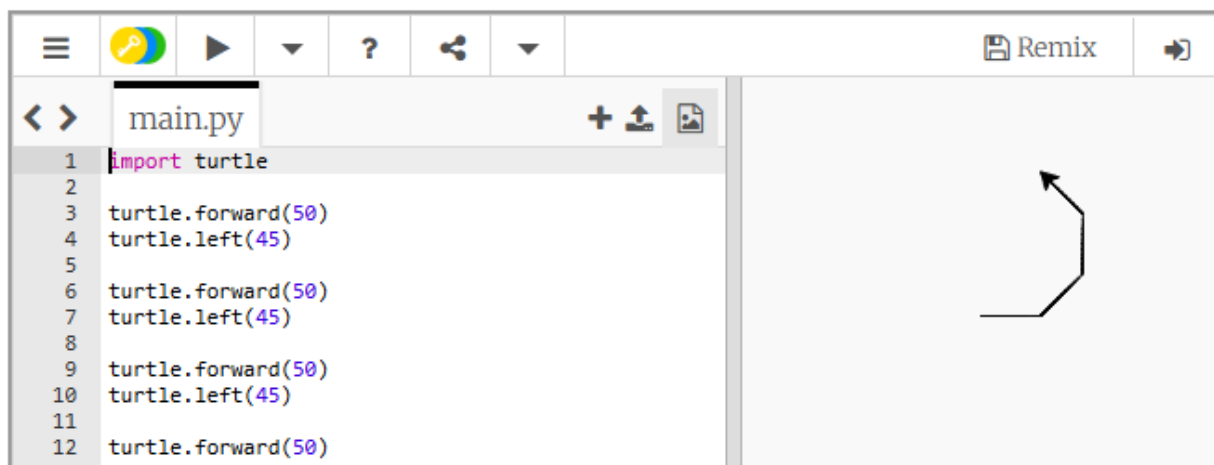
turtle.forward(50)
turtle.left(45)

turtle.forward(50)
turtle.left(45)

turtle.forward(50)
turtle.left(45)

turtle.forward(50)
```

В начале направление черепашки составляет 0° . Далее черепашка перемещается на 50 пикселей вперед и поворачивается на 45° влево. Затем еще дважды повторяет перемещение вперед на 50 пикселей и поворот на 45° влево. После всех этих поворотов угловое направление черепашки составит 135° .



Установка углового направления черепашки

Команда `turtle.setheading()` применяется для установки углового направления черепашки с заданным углом. В качестве аргумента нужно указать желаемый угол.

Рассмотрим код:

```
import turtle

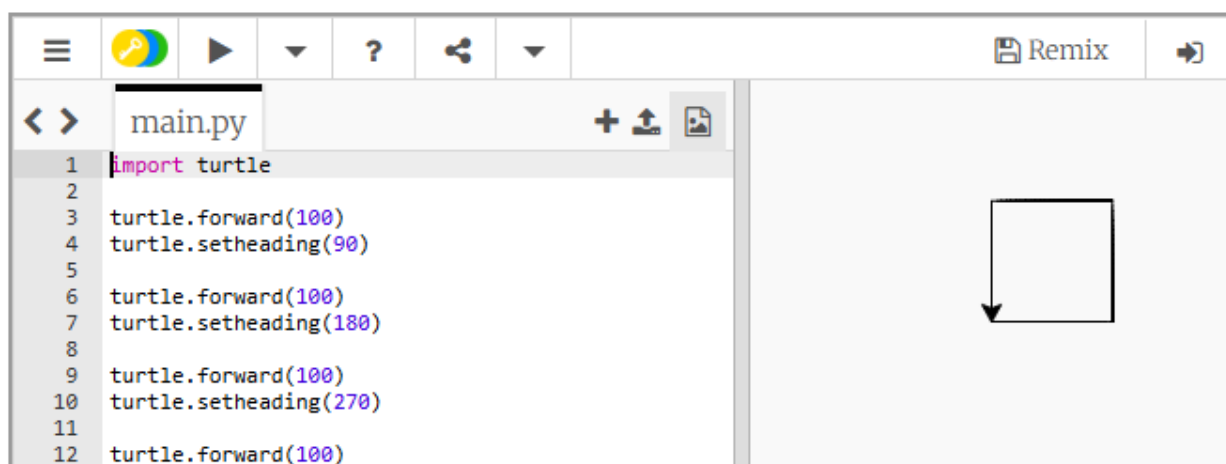
turtle.forward(100)
turtle.setheading(90)

turtle.forward(100)
turtle.setheading(180)

turtle.forward(100)
turtle.setheading(270)

turtle.forward(100)
```

Первоначальное угловое направление черепашки составляет 0° . Далее установлено направление черепашки 90° (на север). Затем установлено направление черепашки 180° (на запад). Потом установлено направление черепашки 270° (на юг).



Получение текущего углового направления черепашки

Чтобы получить текущее угловое направление черепашки используется команда `turtle.heading()` .

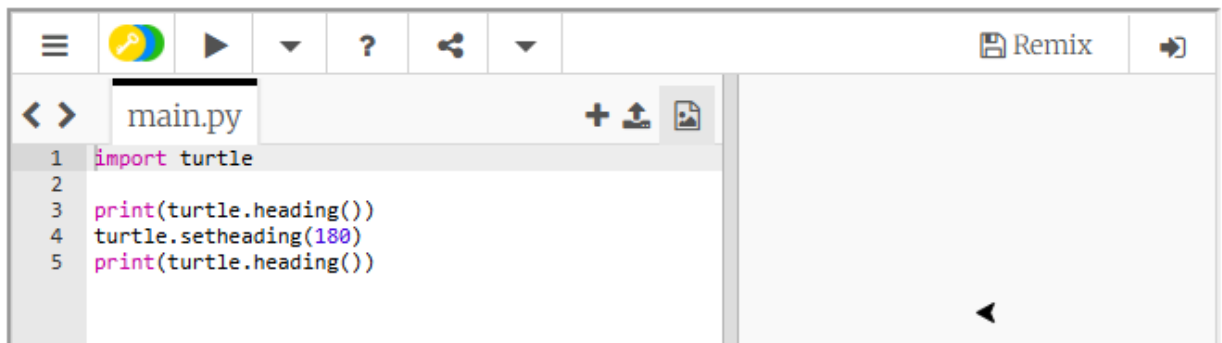
Приведенный ниже код:

```
import turtle

print(turtle.heading())
turtle.setheading(180)
print(turtle.heading())
```

поворачивает черепашку на запад и выводит:

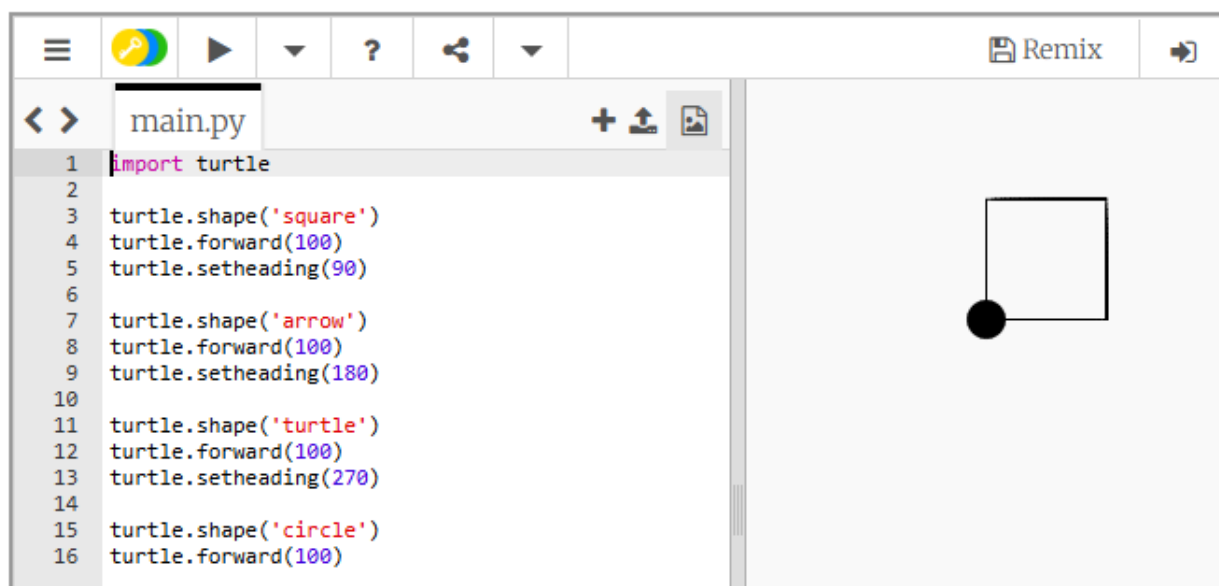
```
0.0
180.0
```



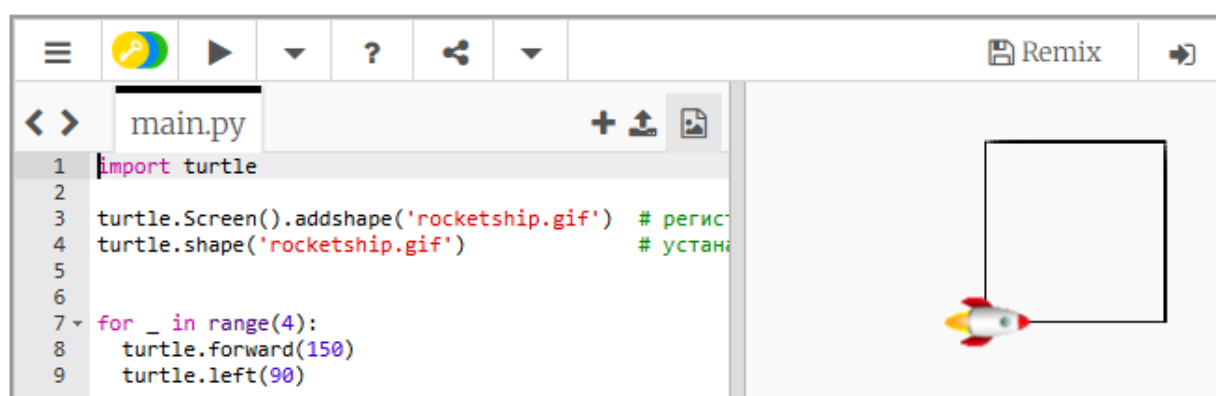
Изменение внешнего вида черепашки

По умолчанию черепашка выглядит как стрелочка, но возможен и другой внешний вид. Для его изменения используют команду `shape()` . Команда принимает в качестве аргумента строковое название фигуры, определяющей форму черепашки. Доступные фигуры:

- square (квадрат);
- arrow (стрелка);
- circle (круг);
- turtle (черепашка);
- triangle (треугольник);
- classic (классическая стрелка).



При необходимости можно использовать собственные рисунки для создания внешнего вида черепашки.

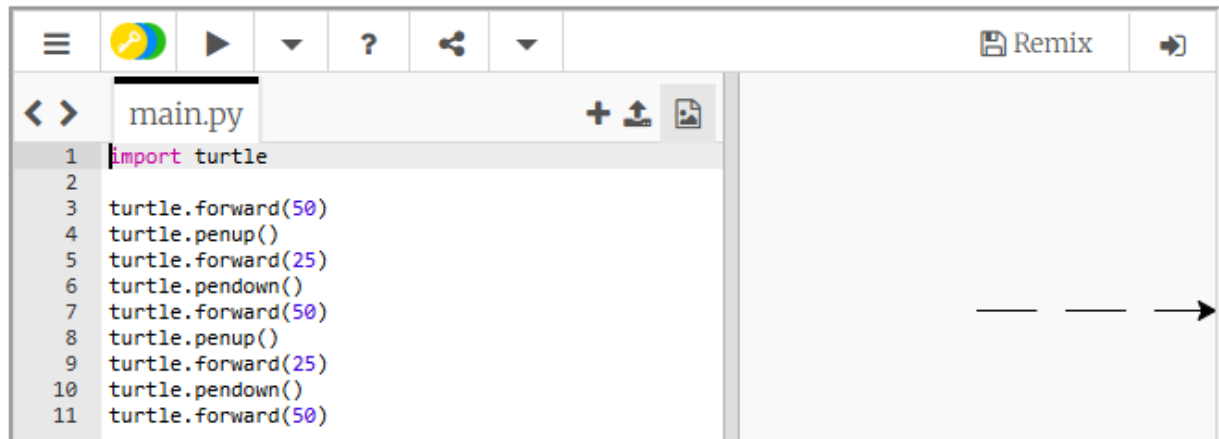


Поднятие и опускание пера

Исходная роботизированная черепашка, использованная преподавателем Сеймуром Пейпертом, лежала на большом листе бумаги и имела перо, которое можно было поднимать и опускать. Когда перо было опущено на бумагу, оно чертило линию во время перемещения черепашки. Когда перо поднималось, оно не касалось бумаги, и черепашка во время движения не оставляла за собой линии.

Команда `turtle.penup()` поднимает перо, а команда `turtle.pendown()` – опускает. Когда перо поднято, черепашка перемещается не оставляя линии. Когда перо опущено, черепашка во время перемещения чертит линию. По умолчанию перо опущено.

выводит пунктирную линию.

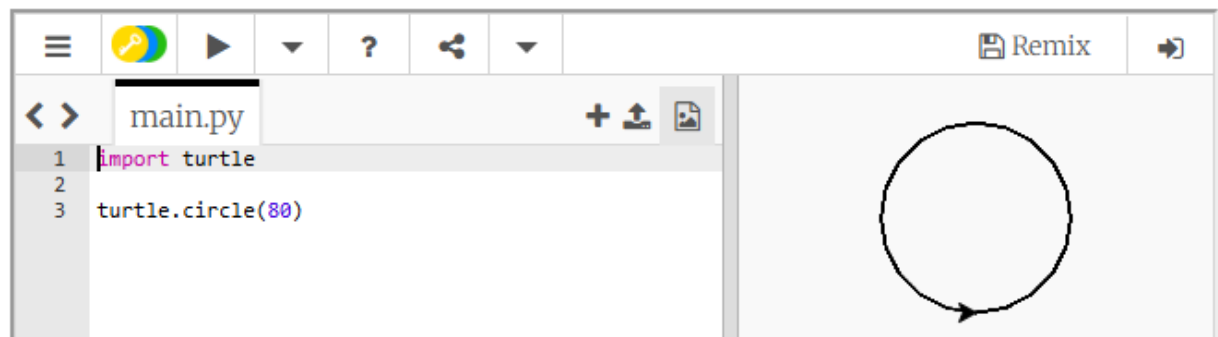


Рисование кругов и точек

Чтобы черепашка нарисовала круг, можно применить команду `turtle.circle()`. Такая команда рисует круг заданного в виде аргумента в пикселях радиуса.

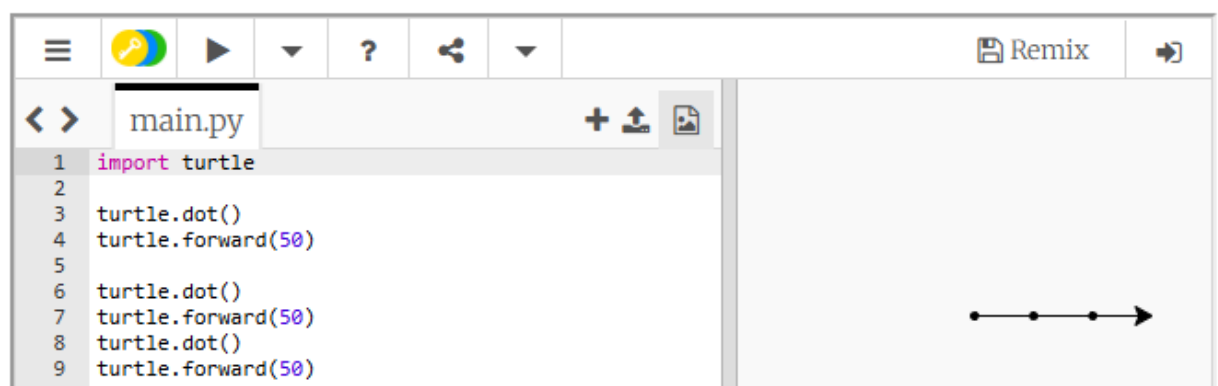
Например, команда `turtle.circle(80)` побуждает черепашку нарисовать круг с радиусом 80 пикселей.

изображает круг радиусом в 80 пикселей.



Команда `turtle.dot()` применяется, чтобы черепашка нарисовала точку.

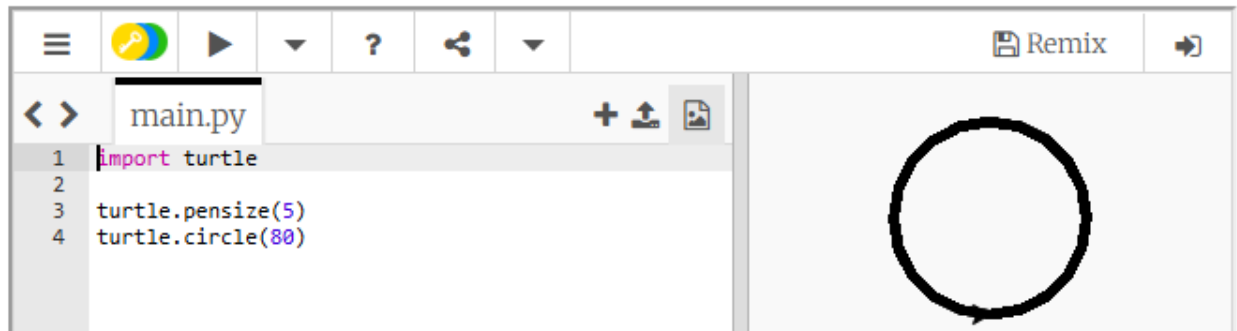
рисует три точки на одной прямой с расстоянием в 50 пикселей между ними.



Изменение размера пера

Для изменения ширины пера черепашки в пикселях можно применить команду `turtle.pensize()`. Аргумент команды – целое число, задает ширину пера.

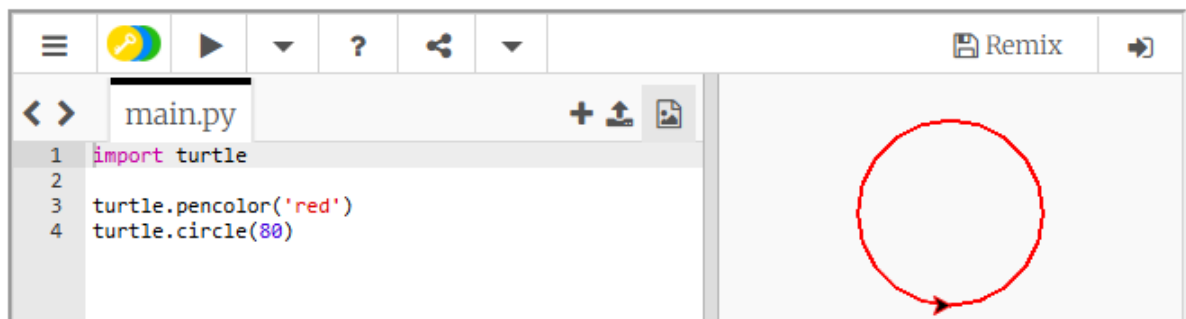
Приведенный ниже код устанавливает ширину пера в 5 пикселей и затем рисует круг:



Изменение цвета рисунка

Для изменения цвета рисунка можно применить команду `turtle.pencolor()`. Аргумент команды – строковое представление названия цвета.

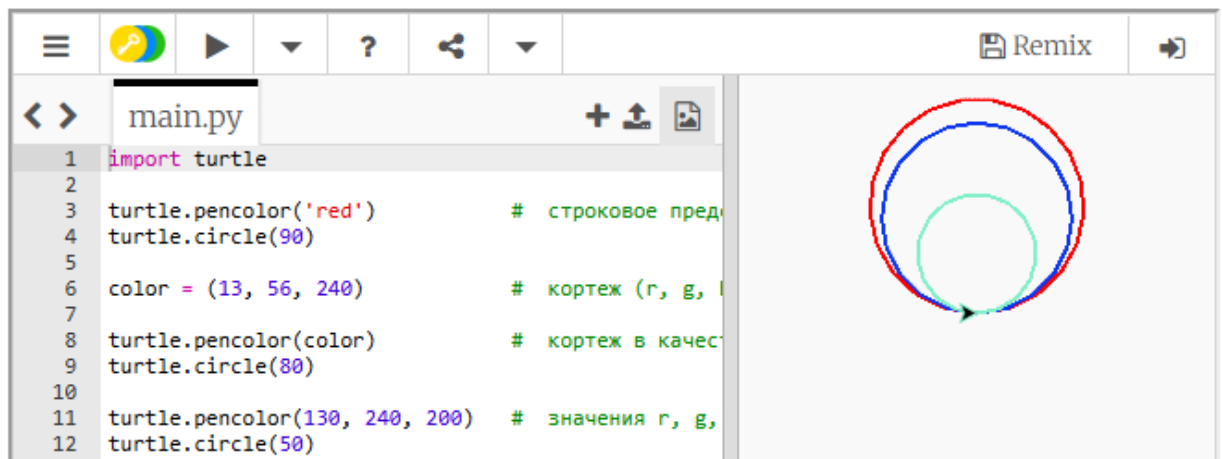
Приведенный ниже код меняет цвет рисунка на красный и затем рисует круг:



С командой `turtle.pencolor()` можно использовать многочисленные predefined названия цветов. Наиболее широко используемые названия цветов:

- red (красный);
- green (зеленый);
- blue (синий);
- yellow (желтый);
- cyan (сине-зелёный).

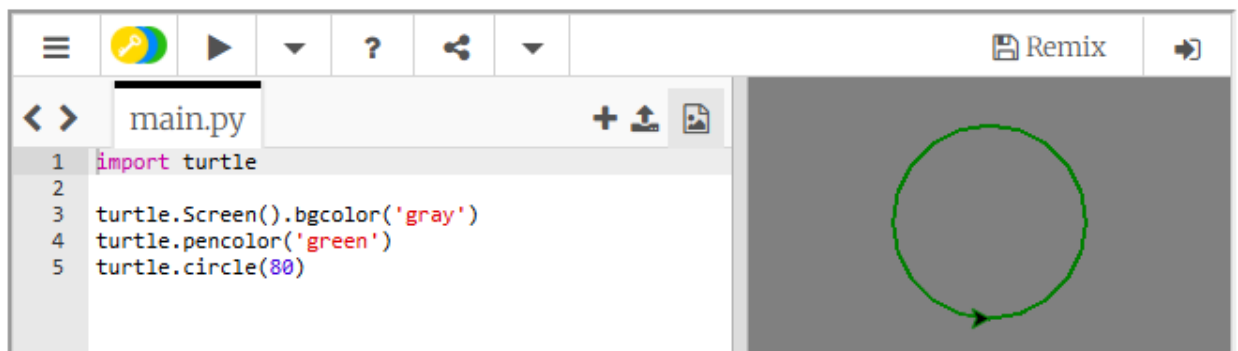
Команда `turtle.pencolor()` позволяет работать не только с predefined названиями цветов, но и с цветами, заданными в формате RGB (Red Green Blue). В качестве аргумента команды `turtle.pencolor()` можно использовать либо кортеж из 3 чисел `(r, g, b)`, либо просто три числа `r, g, b`.



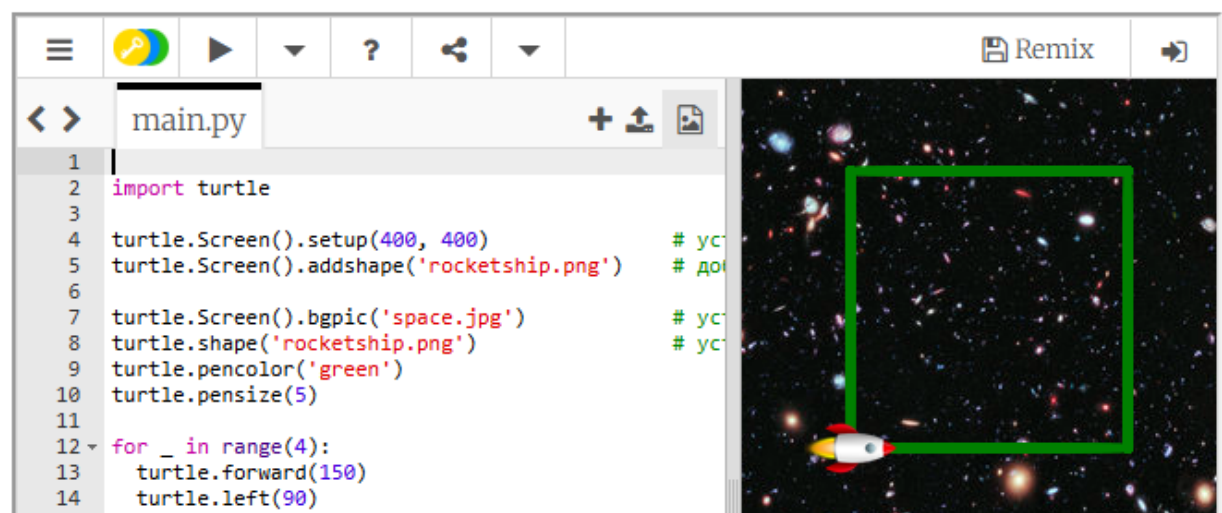
Изменение цвета фона

Для изменения фонового цвета графического окна можно применить команду `turtle.Screen().bgcolor()`. В этом случае тоже можно использовать цвета с predefined названиями или задать цвет в RGB формате.

Приведенный ниже код меняет цвет фона на серый, цвет рисунка на зеленый и затем рисует круг:



Мы также можем установить фоновое изображение с помощью команды `turtle.Screen().bgpic()`.

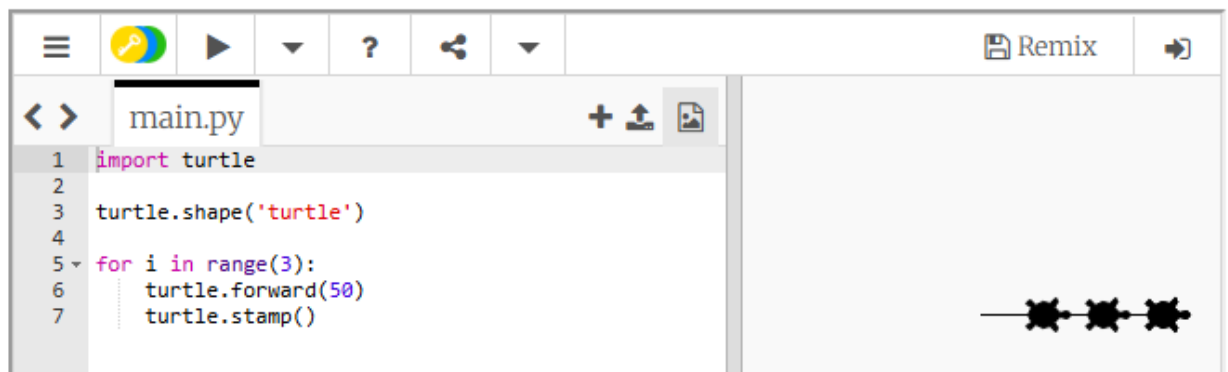


Создание штампа

С помощью команды `turtle.stamp()` можно оставить штамп черепашки. Использование команды `turtle.stamp()` производит тот же эффект, что и команда `turtle.dot()`, но оставляет отметку в форме черепашки.

Приведенный ниже код:

При использовании команды `turtle.stamp()` цветом штампа будет цвет заливки или по умолчанию черный.



Возвращение экрана в исходное состояние

Для возвращения графического окна черепашки в исходное состояние существуют три команды:

- `turtle.clear()` ;
- `turtle.reset()` ;
- `turtle.clearscreen()` .

Команда `turtle.clear()` стирает все рисунки в графическом окне. Но не меняет положение черепашки, цвет рисунка и цвет фона графического окна.

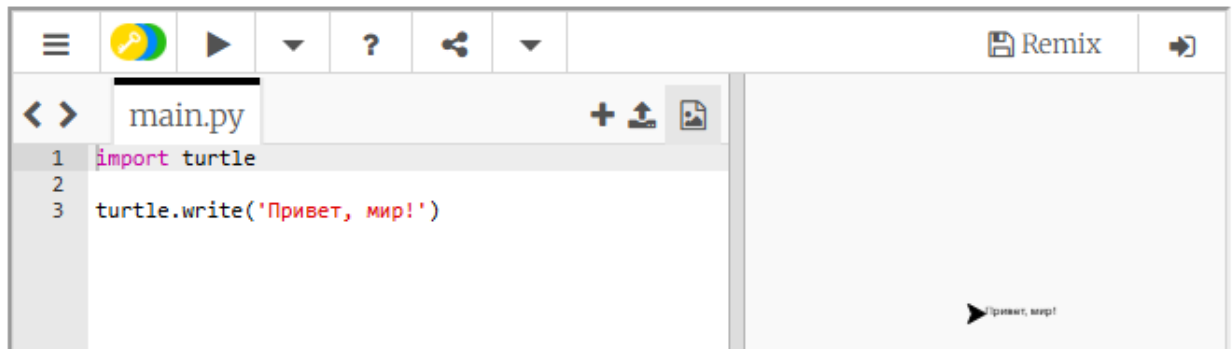
Команда `turtle.reset()` стирает все рисунки, имеющиеся в графическом окне, задает черный цвет рисунка и возвращает черепашку в исходное положение в центре экрана. Эта команда не переустанавливает цвет фона графического окна.

Команда `turtle.clearscreen()` стирает все рисунки в графическом окне, меняет цвет рисунка на черный, а цвет фона на белый, и возвращает черепашку в исходное положение в центре графического окна.

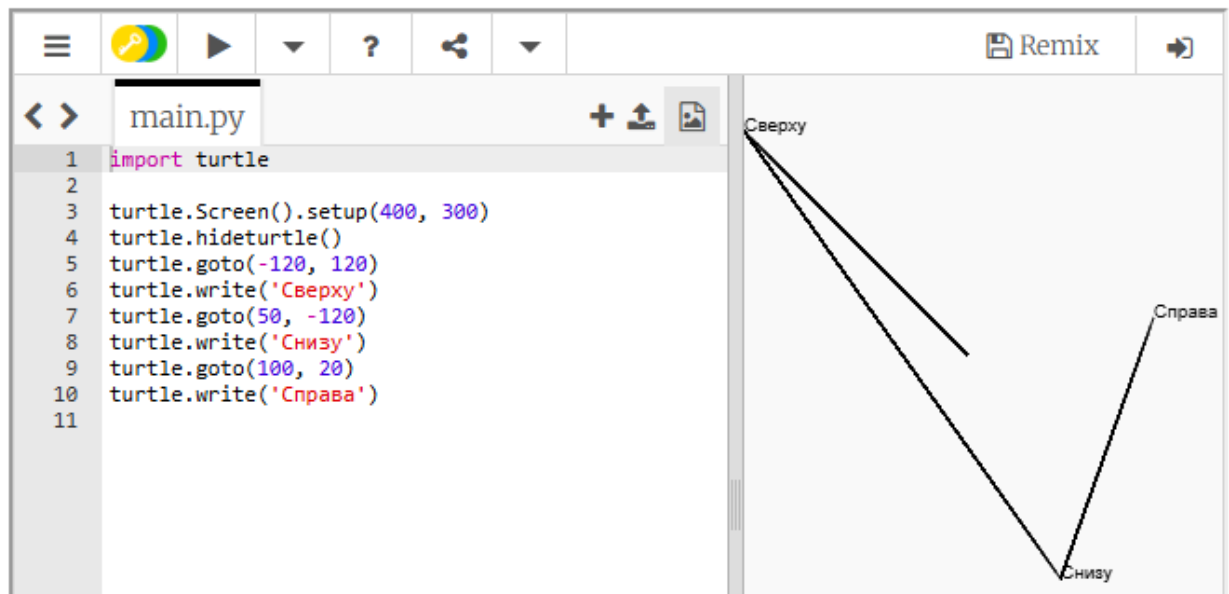
Вывод текста в графическое окно

Для вывода текста в графическое окно применяется команда `write()`. Аргумент этой команды — строка текста, которую требуется вывести на экран. Левый нижний угол первого символа выведенного текста будет расположен в точке с координатами черепашки.

выводит текст `Привет, мир!` на экран.



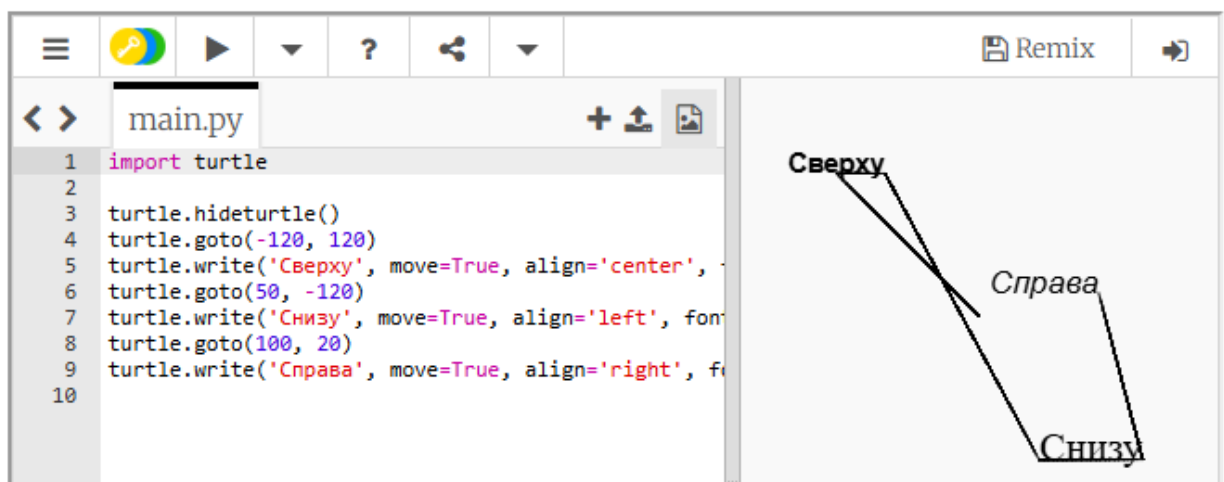
Приведенный ниже код показывает, как черепаха перемещается в соответствующие позиции для вывода текста:



Мы можем настроить вывод текста, задавая размер и тип шрифта, начертание, выравнивание и т.д.

Аргументы команды `write()` :

- `arg` – текст, который нужно вывести;
- `move` – указывает будет ли двигаться черепашка по мере рисования надписи (по умолчанию значение `False`);
- `align` – служит для выравнивания надписи относительно черепашки, может принимать три строковых значения `right`, `center`, `left` (по умолчанию значению `right`);
- `font` – кортеж из трех значений: (название шрифта, размер шрифта, тип начертания) . В качестве начертания можно использовать строковые значения: `normal` – обычный, `bold` – полужирный, `italic` – курсив, или объединить два последних, тогда текст будет напечатан полужирным курсивом.

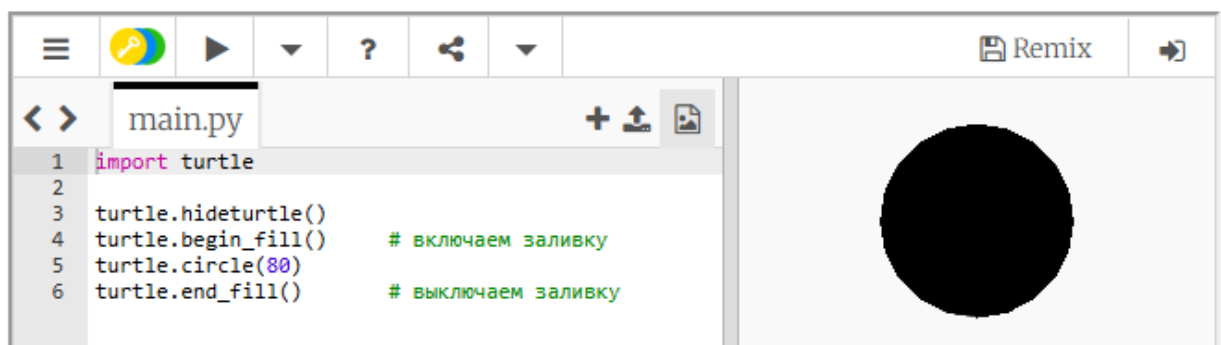


Заполнение геометрических фигур

Для заполнения геометрической фигуры цветом используется команда `turtle.begin_fill()` , причем она применяется до начертания фигуры, а после завершения начертания используется команда `turtle.end_fill()` и геометрическая фигура заполняется текущим цветом заливки.

Приведенный ниже код:

рисует круг, заполненный цветом по умолчанию – черным.

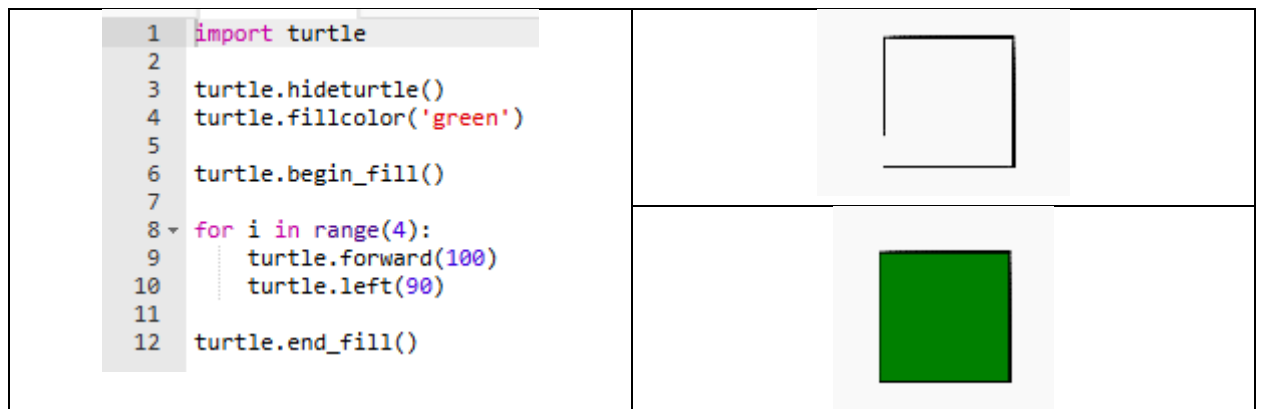
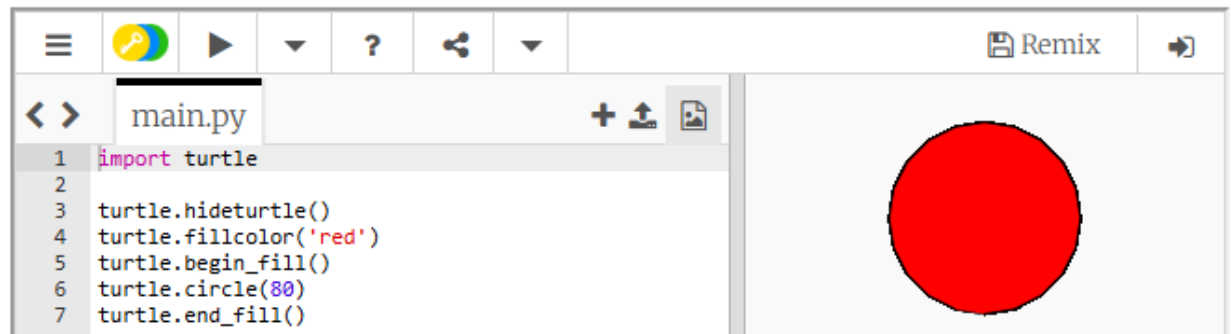


Цвет заливки можно изменить при помощи команды `fillcolor()`.

Аргумент команды — название цвета в виде строкового литерала, либо значения трех компонентов RGB.

Приведенный ниже код:

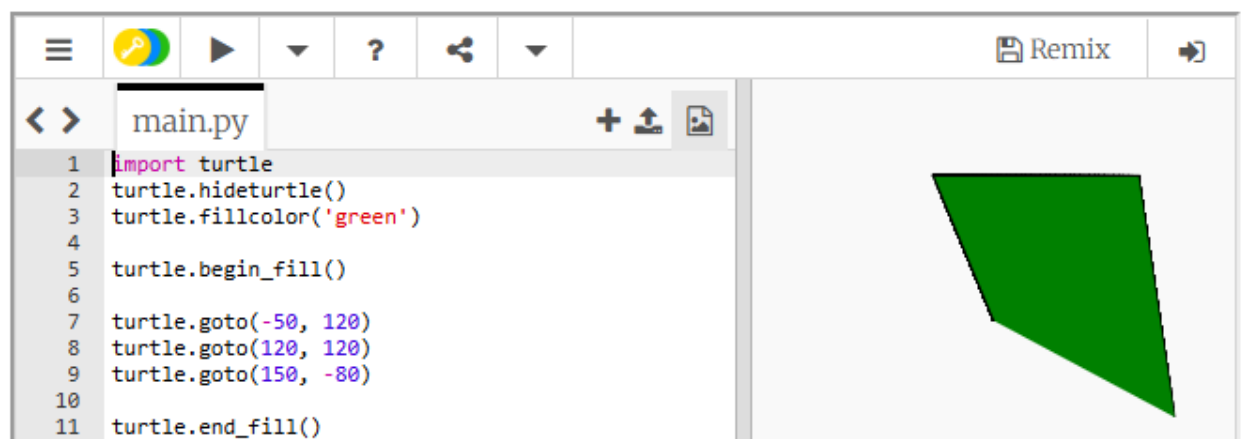
изменяет цвет рисунка на красный и рисует круг.



Если заполнить незамкнутую фигуру, она будет закрашена, будто был начерчен отрезок, соединяющий начальную и конечную точки.

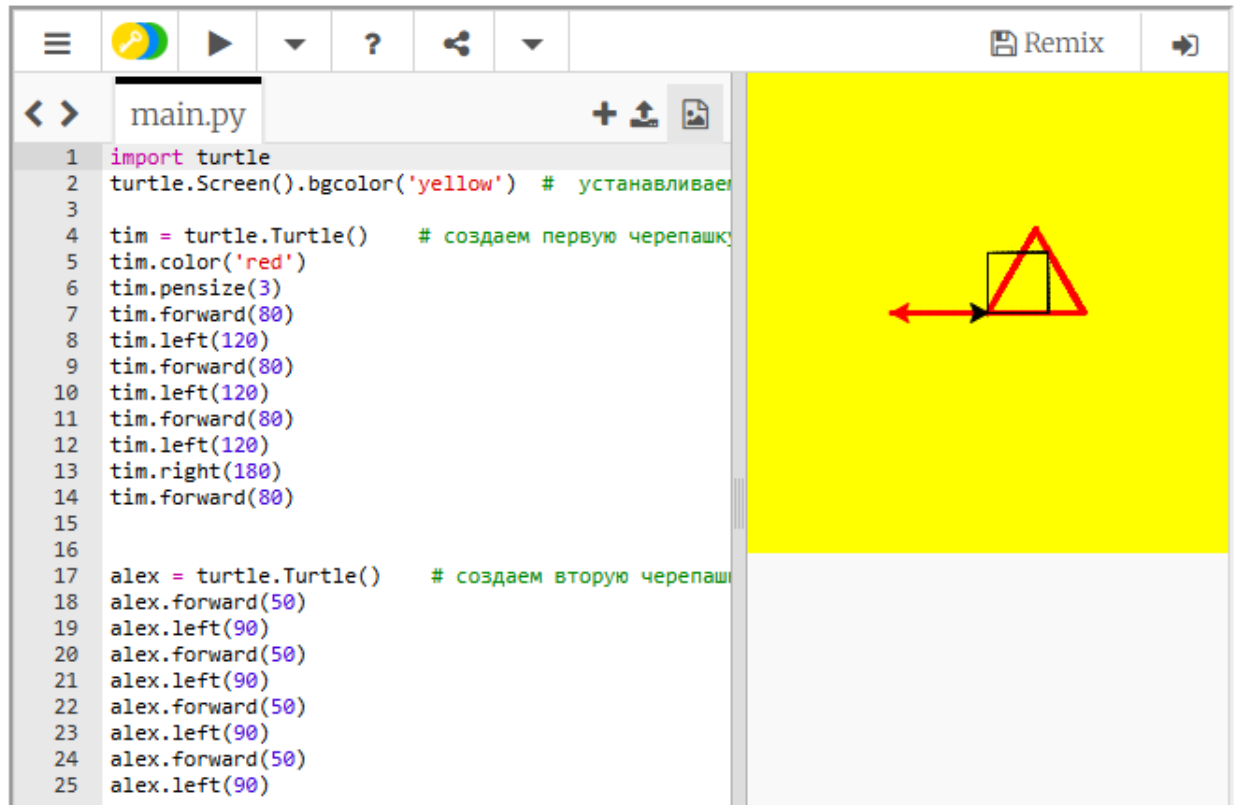
Приведенный ниже код:

рисует зеленый четырехугольник.



Создание нескольких черепашек

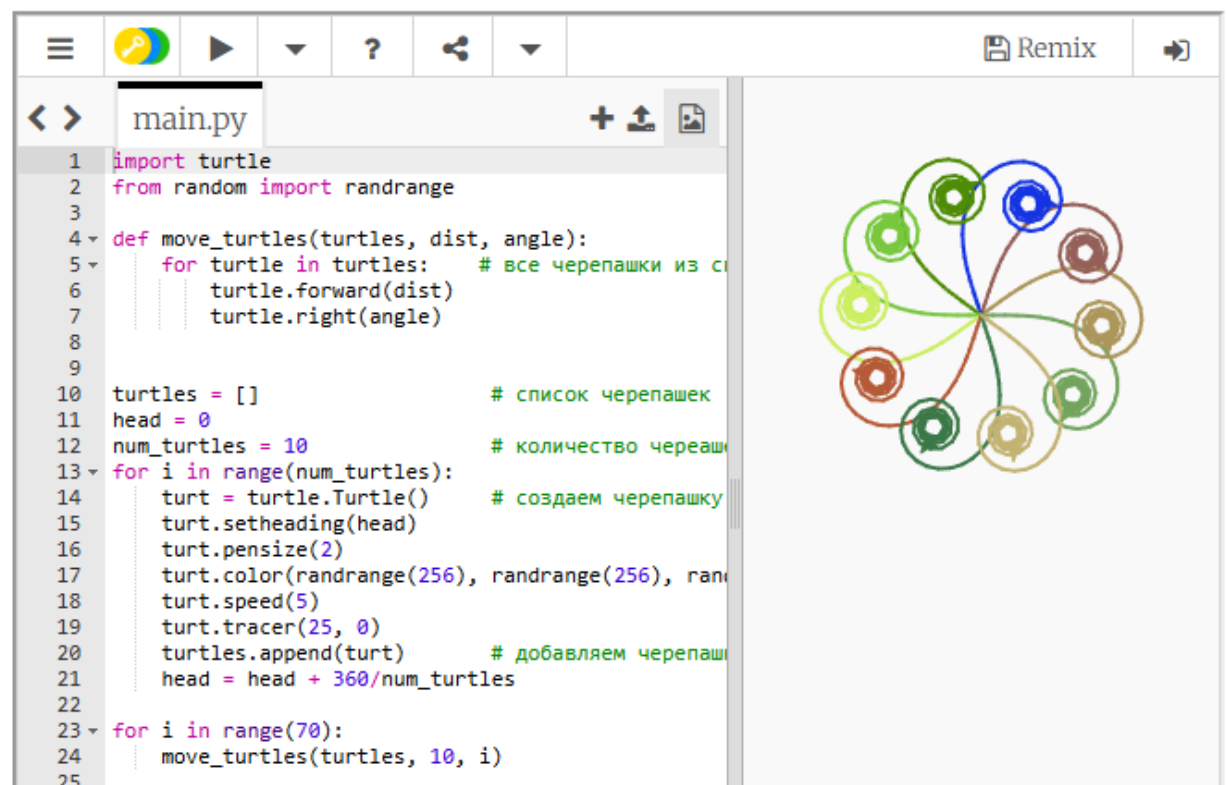
Можно работать сразу с несколькими черепашками. Для этого надо создать несколько переменных, содержащих экземпляры класса `Turtle`.



```
1 import turtle
2 turtle.Screen().bgcolor('yellow') # устанавливаем
3
4 tim = turtle.Turtle() # создаем первую черепашку
5 tim.color('red')
6 tim.pensize(3)
7 tim.forward(80)
8 tim.left(120)
9 tim.forward(80)
10 tim.left(120)
11 tim.forward(80)
12 tim.left(120)
13 tim.right(180)
14 tim.forward(80)
15
16
17 alex = turtle.Turtle() # создаем вторую черепашку
18 alex.forward(50)
19 alex.left(90)
20 alex.forward(50)
21 alex.left(90)
22 alex.forward(50)
23 alex.left(90)
24 alex.forward(50)
25 alex.left(90)
```

The screenshot shows a Python IDE with a yellow background. On the left, a code editor displays a script that creates two turtles. The first turtle, 'tim', is red and draws a triangle. The second turtle, 'alex', is black and draws a square. On the right, the canvas shows the resulting drawing: a red triangle and a black square on a yellow background.

Черепашек можно сохранить в списке, а затем в цикле обрабатывать его.



```
1 import turtle
2 from random import randrange
3
4 def move_turtles(turtles, dist, angle):
5     for turtle in turtles: # все черепашки из списка
6         turtle.forward(dist)
7         turtle.right(angle)
8
9
10 turtles = [] # список черепашек
11 head = 0
12 num_turtles = 10 # количество черепашек
13 for i in range(num_turtles):
14     turt = turtle.Turtle() # создаем черепашку
15     turt.setheading(head)
16     turt.pensize(2)
17     turt.color(randrange(256), randrange(256), randrange(256))
18     turt.speed(5)
19     turt.tracer(25, 0)
20     turtles.append(turt) # добавляем черепашку в список
21     head = head + 360/num_turtles
22
23 for i in range(70):
24     move_turtles(turtles, 10, i)
25
```

The screenshot shows a Python IDE with a white background. On the left, a code editor displays a script that creates a list of 10 turtles and draws a complex geometric pattern. The script uses a function 'move_turtles' to move each turtle in a list by a distance of 10 units at an angle of 'i' degrees. The turtles are colored randomly and have a pen size of 2. On the right, the canvas shows the resulting drawing: a complex geometric pattern consisting of many small circles and lines, each colored differently, arranged in a circular pattern.

Отслеживание нажатия клавиш

Черепашья графика позволяет отслеживать происходящие события, такие как нажатия клавиш клавиатуры, перемещение мышки или нажатие на мышку.

Изначально программа никак не реагирует на эти события и чтобы это поведение изменить необходимо привязать функции обратного вызова к событиям. Для этого существуют специальные команды. Для отслеживания нажатия клавиш клавиатуры используется команда `onkey(fun, key)`, которая связывает функцию обратного вызова `fun` с событием нажатия клавиши `key`.

```
import turtle

def move_up():          # функция обратного вызова
    x, y = turtle.pos()
    turtle.setposition(x, y + 5)

def move_down():        # функция обратного вызова
    x, y = turtle.pos()
    turtle.setposition(x, y - 5)

def move_left():        # функция обратного вызова
    x, y = turtle.pos()
    turtle.setposition(x - 5, y)

def move_right():       # функция обратного вызова
    x, y = turtle.pos()
    turtle.setposition(x + 5, y)

turtle.showturtle()     # отображаем черепашку
turtle.pensize(3)       # устанавливаем размер пера
turtle.shape('turtle')

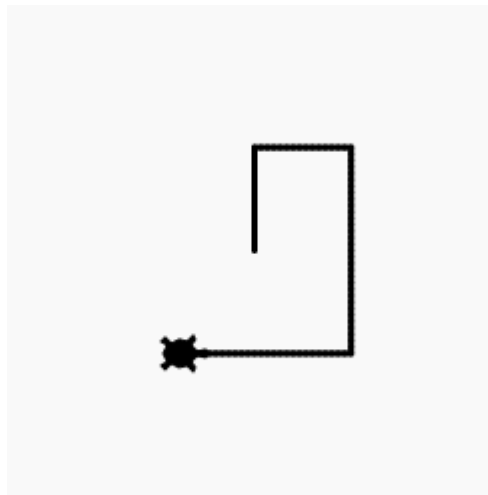
turtle.Screen().listen() # устанавливаем фокус на экран черепашки

turtle.Screen().onkey(move_up, 'Up')  # регистрируем функцию на нажатие клавиши
наверх

turtle.Screen().onkey(move_down, 'Down') # регистрируем функцию на нажатие
клавиши вниз

turtle.Screen().onkey(move_left, 'Left') # регистрируем функцию на нажатие клавиши
налево
```

```
turtle.Screen().onkey(move_right, 'Right') # регистрируем функцию на нажатие клавиши направо
```



Отслеживание нажатия мыши

Аналогичным образом можно отслеживать нажатие на мышку. Для отслеживания нажатия мыши используется команда `onclick(fun)`, которая связывает функцию обратного вызова `fun` с событием нажатия левой кнопки мыши.

Приведенный ниже код рисует круг по нажатию на левую кнопку мыши.

```
1 import turtle
2 from random import randrange
3
4 def random_color():
5     return randrange(256), randrange(256), randrange(256)
6
7 def draw_circle(x, y, r):
8     turtle.penup()
9     turtle.goto(x, y - r)
10    turtle.pendown()
11    color = random_color()
12    turtle.pencolor(color)
13    turtle.fillcolor(color)
14    turtle.begin_fill()
15    turtle.circle(r)
16    turtle.end_fill()
17    turtle.speed(0)
18
19 def left_mouse_click(x, y):
20     draw_circle(x, y, 10)
21
22 turtle.hideturtle()
23
24 turtle.Screen().onclick(left_mouse_click)
25 turtle.Screen().listen()
```

